

Apigee X Simulator

Local Test Guide & Test Data (v8)

This guide lets you reproduce, on your own machine, the same scenarios covered in the automated test run - but this time through the real running app (frontend + backend + a real SQLite database via Prisma), not a sandbox substitute.

What was actually tested before, and what this closes

The 43-case test pass called the real policy execution functions in `backend/src/Utils/policyExecutors.ts` directly - the same functions `traceEngine.ts` calls during a real Trace run - so the actual policy logic (header parsing, JWT/issuer/audience/expiry checks, rate-limit math, cache round-trips, basic-auth encode/decode, route-rule selection, etc.) was genuinely exercised, not guessed at.

What that pass could **not** cover: it ran in a sandbox that can't reach Prisma's engine download host, so a real database was substituted with an in-memory stand-in. That means the full path - real HTTP request in the browser, real Trace API call, real SQLite reads/writes, real UI rendering - was not personally verified end-to-end by me in that pass. This guide is how you close that gap yourself, using the exact same scenarios and the exact same expected outcomes.

Setup checklist

- Unzip `Apigee-X-Simulator-v8.zip`.
- Backend: `cd backend && npm install`.
- If upgrading from an older zip: delete `prisma/dev.db` first (schema changed - `targetEndpoint` is now `targetEndpoints`, plus a new `IssuedToken` table).
- `npx prisma migrate dev` (any migration name), then `npx prisma db seed` if it doesn't run automatically.
- `npm run dev` in backend (default port 4000).
- Frontend: `cd frontend && npm install && npm run dev`.
- Log in with `admin@apigee-sim.local` / `Apigee123!`.

Trace Page Primer

Every test case below is run from the **Trace** page. Fields you'll use: **Proxy** (dropdown), **Environment**, **Method**, **Path** (query params can be typed directly, e.g. /secure?access_token=abc), **Headers** (key/value rows - case doesn't matter any more, that was the 401 bug), **Request Body** (JSON editor), and **Run Trace**. After running, check the **Flow Execution Timeline** (each policy's pass/fail), the **Response** panel, and **Variables Created**.

Security Policies

1. VerifyAPIKey - valid key

Proxy: ProductCatalog-v1 | Environment: test or prod

Policy config: apiKeyLocation=header, apiKeyParam=x-api-key (policy name VA-VerifyKey).

Steps:

- 1 Go to Apps, open any app (e.g. Acme-Mobile-App), copy its Consumer Key.
- 2 Trace: Method GET, Path /products, Header x-api-key: <consumer key>, Run Trace.

Expected result:

- Timeline shows VA-VerifyKey = success.
- Variables include verifyapikey.client_id, verifyapikey.developer.app.name, verifyapikey.developer.email.
- Overall response is 200 (product data from the real target).

2. VerifyAPIKey - missing / invalid key

Proxy: ProductCatalog-v1

Steps:

- 1 Trace: GET /products with no x-api-key header at all -> Run Trace.
- 2 Repeat with header x-api-key: not-a-real-key.

Expected result:

- Both attempts: VA-VerifyKey = error, overall response status 401.
- Missing case faultstring mentions "Failed to resolve API Key variable"; invalid case says "Invalid API Key".

3. OAuthV2 - GenerateAccessToken

Proxy: OAuthToken-v1 (POST /oauth/token)

Policy OA-GenerateToken: operation=GenerateAccessToken, grantType=client_credentials, scopes=read.

Steps:

- 1 Go to Apps, open any app, copy Consumer Key and Consumer Secret.
- 2 Trace: Method POST, Path /oauth/token, Body: {"client_id":"<key>","client_secret":"<secret>"}, Run Trace.

Expected result:

- Response body contains access_token, token_type="Bearer", expires_in=3600.
- Variables include oauthv2.access_token and oauthv2.developer.app.name.
- Copy the access_token value - you'll use it in the next test.

4. OAuthV2 - VerifyAccessToken (Header, default)

Proxy: OAuthProtected-v1 (GET /secure)

Policy OA-VerifyToken: operation=VerifyAccessToken, accessTokenLocation=Header (default).

Steps:

- 1 Trace: Method GET, Path /secure, Header Authorization: Bearer <access_token from test 3>, Run Trace.

Expected result:

- OA-VerifyToken = success; this is the exact bug you reported - it now passes because header keys are normalized to lowercase before any policy reads them.
- Variables include oauthv2.client_id and oauthv2.scope.

5. OAuthV2 - VerifyAccessToken via Query Parameter (new feature)

Proxy: OAuthProtected-v1

Steps:

- 1 Open OAuthProtected-v1 > Develop tab > click policy OA-VerifyToken.
- 2 Configuration tab: set Access Token Location = "Query Parameter", Query Param Name = access_token. Save.
- 3 Trace: Method GET, Path /secure?access_token=<access_token from test 3>, no Authorization header, Run Trace.

Expected result:

- OA-VerifyToken = success, token read from the query string instead of a header.
- This is the exact capability you asked for - "pass the verify access token in the auth header or query parameter".

6. OAuthV2 - failure paths

Proxy: OAuthProtected-v1 / OAuthToken-v1

Steps:

- 1 Trace GET /secure with no Authorization header and no query param.
- 2 Trace GET /secure with Authorization: Bearer not-a-real-token.
- 3 Trace POST /oauth/token with the right client_id but a wrong client_secret.

Expected result:

- Case 1: 401, faultstring "Invalid Access Token", message mentions the header/query param it expected.
- Case 2: 401, "Access token is invalid or was never issued by this simulator".
- Case 3: 401, error "invalid_client".

7. VerifyJWT - valid, expired, wrong issuer, wrong algorithm

Proxy: OrderService-v1 (GET/POST /orders)

Policy VerifyJWT default config: algorithm=RS256, issuer=https://accounts.example.com, audience=my-api.

Steps:

- 1 Trace: Method GET, Path /orders, Header Authorization: Bearer <GOOD JWT from Appendix A>, Run Trace -> expect success.
- 2 Repeat using the EXPIRED JWT -> expect 401 "JWT has expired".
- 3 Repeat using the WRONG_ISSUER JWT -> expect 401 "JWT issuer is invalid".
- 4 Repeat using the WRONG_ALG JWT -> expect 401 "JWT algorithm mismatch".

Expected result:

- Good token: VerifyJWT = success, variables jwt.decoded.header / jwt.decoded.claims populated.
- Each bad token: VerifyJWT = error, 401, with the specific faultstring noted above.
- Note: this simulator checks issuer/audience/algorithm/expiry from the decoded claims - it does not cryptographically verify the signature - so the ready-made tokens in Appendix A work without a real private key.

8. BasicAuthentication - Decode and Encode

Not attached to any seeded proxy - attach it temporarily to test standalone.

Steps:

- 1 Open CustomerService-v1 > Develop tab > Policies panel > Add, choose BasicAuthentication.
- 2 Configuration: operation=Decode, User Variable=req.user, Password Variable=req.pass. Save (attaches to ProxyEndpoint PreFlow by default).
- 3 Trace: GET /customers, Header Authorization: Basic YWxpY2U6czNjcmV0 (this is base64 for alice:s3cret), Run Trace.
- 4 Remove the policy afterwards (trash icon) to restore the proxy to its original state.

Expected result:

- Decode: policy = success, variables req.user="alice" set (password variable is masked to "*****" in the demo executor's message).
- No/invalid Basic header -> 401 "Invalid basic authentication header".

Traffic Management

9. CORS headers

Proxy: WeatherProxy-v1 (GET /weather)

Policy CORS-Allow with default allow-all config.

Steps:

- 1 Trace: GET /weather, any environment, Run Trace.

Expected result:

- Response headers include Access-Control-Allow-Origin, -Methods, -Headers.
- CORS-Allow shows success in the timeline.

10. SpikeArrest - trigger a 429

Proxy: CustomerService-v1 (SA-Ratelimit, default 50ps)

Steps:

- 1 Open CustomerService-v1 > SA-Ratelimit policy > temporarily set rate to 3ps > Save.
- 2 Trace GET /customers four or five times in a row, quickly (within about a second).

Expected result:

- First 3 calls: SA-Ratelimit = success.
- 4th/5th call: SA-Ratelimit = error, 429, "Spike arrest violation".
- Restore rate to 50ps afterwards.

11. Quota - exceed the allowed count

Proxy: OrderService-v1 (Quota-Limit, default 1000/hour)

Steps:

- 1 Open OrderService-v1 > Quota-Limit policy > temporarily set Allow Count to 3, interval 1 hour > Save.
- 2 Trace GET /orders four times.

Expected result:

- First 3 calls: Quota-Limit = success, ratelimit.Quota.count increments 1, 2, 3.
- 4th call: error, 429, "Quota Violation".
- Restore Allow Count to 1000 afterwards.

12. ResponseCache - second call served from cache

Proxy: ProductCatalog-v1 (RC-ProductCache, ttl 120s)

Steps:

- 1 Trace GET /products (test or prod environment). Note the timeline and response.
- 2 Immediately Trace GET /products again with the same path.

Expected result:

- First call: responsecache.cachehit = false, real target call happens.
- Second call: responsecache.cachehit = true, Target Request step shows "skipped - response served from cache".

13. Target Failover

Proxy: FailoverDemo-v1 (test environment only)

Primary target LegacyBillingService points at a deliberately unreachable host.

Steps:

- 1 Trace: GET /failover-demo, environment test, Run Trace.

Expected result:

- Timeline shows a failed HTTP Call to LegacyBillingService, then "HTTP Call (Failover)" to ProductService succeeding.
- Overall trace still succeeds (failover worked).

Routing & Multi-Method Features (this session's main ask)

14. Route Rules - GET vs POST/PUT/PATCH/DELETE to different targets

Proxy: MultiRouteDemo-v1 (test environment, base path /items)

Route rule "reads" (verb=GET) -> target endpoint read-target (ProductService); fallback rule "writes" -> write-target (CustomerService).

Steps:

- 1 Trace: Method GET, Path /items, environment test, Run Trace.
- 2 Trace: Method POST, Path /items, same environment, Run Trace.
- 3 Trace: Method DELETE, Path /items, Run Trace.

Expected result:

- GET: Routing step says routed to "read-target"; target call hits ProductService's host.
- POST and DELETE: Routing step says routed to "write-target"; target call hits CustomerService's host.
- This proves one proxy is genuinely routing different HTTP methods to different target servers.

15. Editing Target Endpoints and Route Rules in the UI

Proxy: MultiRouteDemo-v1 > Develop tab

Steps:

- 1 Open the proxy, Develop tab. You'll see a Target Endpoints panel (read-target, write-target) and a Route Rules table.
- 2 Click "Add Target Endpoint", name it e.g. patch-target, point it at a Target Server.
- 3 In Route Rules, click "Add Route Rule", set Verb=PATCH, Target Endpoint=patch-target, save.
- 4 Trace: Method PATCH, Path /items -> should now route to patch-target.

Expected result:

- New target endpoint and route rule persist after reload.
- PATCH request in Trace resolves to the new target endpoint's server.

16. Conditional Flows

Any proxy > Develop tab > Conditional Flows panel

Steps:

- 1 Click "Add Conditional Flow", set Verb=POST and/or a Base Path Suffix.
- 2 In the Policies panel, set "Attach new policy to" = the new conditional flow's request, then Add a policy (e.g. AssignMessage).
- 3 Trace a request matching that verb/path.

Expected result:

- Timeline shows a step under "Conditional Flow: <name> (Request)" instead of/in addition to ProxyEndpoint PreFlow.
- A request that does NOT match the condition skips that flow entirely.

Products, Apps & Policy Editing UX

17. Multiple proxies per Product, multiple Products per App

Products page / Apps page

Steps:

- 1 Products: open Free-Tier or Premium-Tier, use the edit icon to add/remove proxy chips.
- 2 Apps: open any app, use the edit icon to add/remove Product associations.

Expected result:

- Changes save and persist on reload; an app can hold multiple products, sharing one set of credentials across all of them.

18. XML view vs Configuration (read this if a policy "isn't taking effect")

Any policy > PolicyConfigDialog

Steps:

- 1 Open any policy, look at the XML View tab.

Expected result:

- The banner now says explicitly that Trace only ever executes the Configuration tab's values - hand-edited XML is for reference/export only.
- If you need a behavior change (like OAuthV2's token location), it must be a real field on the Configuration tab - which is exactly what was added for the token-location request.

Appendix A - Ready-to-use JWTs for VerifyJWT tests

These match OrderService-v1's default VerifyJWT config (issuer https://accounts.example.com, audience my-api, algorithm RS256). The simulator checks these claims but does not cryptographically verify the signature, so these tokens work as-is.

GOOD (valid for ~1 year)

eyJhbGciOiAiUmlrNTYiLCJhdhWlIjogIkpXVCJ9.eyJpc3MiOiAiaHR0cHM6Ly9hY2NvdW50cy5leGFtcGxlLmNvbSIsICJhdWQiOiAibXktYXBpIiwgInN1YiI6ICh...

EXPIRED

eyJhbGciOiAiUmlrNTYiLCJhdhWlIjogIkpXVCJ9.eyJpc3MiOiAiaHR0cHM6Ly9hY2NvdW50cy5leGFtcGxlLmNvbSIsICJhdWQiOiAibXktYXBpIiwgInN1YiI6ICh...

WRONG ISSUER

eyJhbGciOiAiUmlrNTYiLCJhdhWlIjogIkpXVCJ9.eyJpc3MiOiAiaHR0cHM6Ly9ldmVsLmV4YW1wbGUuY29tIiwgImF1ZCI6IChJteS1hcGkiLCJhdWQiOiAibXktYXBpIiwgInN1YiI6ICh...

WRONG ALGORITHM (HS256 instead of RS256)

eyJhbGciOiAiSFMyNTYiLCJhdhWlIjogIkpXVCJ9.eyJpc3MiOiAiaHR0cHM6Ly9hY2NvdW50cy5leGFtcGxlLmNvbSIsICJhdWQiOiAibXktYXBpIiwgInN1YiI6ICh...

Basic Auth test credential

Base64 of alice:s3cret (used in test 8):
YWxpY2U6czNjcmV0

Appendix B - Seeded Proxies Reference

Proxy	Base Path	Deployed Envs	Key Policies
CustomerService-v1	/customers	dev, test, prod	VerifyAPIKey, SpikeArrest 50ps, AssignMessage
OrderService-v1	/orders	dev, test, prod	VerifyJWT, SpikeArrest 100pm, Quota 1000/hr, AssignMessage
ProductCatalog-v1	/products	test, prod	VerifyAPIKey (x-api-key), Quota 5000/day, ResponseCache 120s
WeatherProxy-v1	/weather	dev, test, prod	CORS, SpikeArrest 20ps
FailoverDemo-v1	/failover-demo	test	SpikeArrest 50ps, target failover
OAuthToken-v1	/oauth/token	dev, test, prod	OAuthV2 GenerateAccessToken
OAuthProtected-v1	/secure	dev, test, prod	OAuthV2 VerifyAccessToken
MultiRouteDemo-v1	/items	test	Route Rules: GET->read-target, others->write-target